Homework 4: Bayesian Inference for Mixture Models

Baijia Xu

2026-04-02

Bayesian K-Mixture Model

We consider a Bayesian Mixture Model of unit-variance univariate Gaussians. There are:

- K mixture components, corresponding to K Gaussian distributions with means $\mu = \{\mu_1, \ldots, \mu_K\}$
- cluster assignments $c_i \in \{1, \ldots, n\}$
- observations $y_i \in \{1, \ldots, n\}$

Data generating model is

$$\begin{aligned} \mu &\sim N(0, \sigma^2) \\ c_i &\sim Categorical(1/K \ldots, 1/K) \\ y_i | c_i, \mu &\sim N(c_i^T \mu, 1) \end{aligned}$$

Problem 1: Gibbs sampler derivations and implementation

Derive a Gibb's sampler for the posterior distribution, $p(\mu, \mathbf{c}|y)$. Show the derivations and resulting Gibb's sampler updates. Specifically, derive:

- $p(c_i|y_i, \mu)$
- $p(\mu_k|y, c)$

Derivations

1. Conditional posterior of cluster assignments $p(c_i|y_i, \mu)$ By Bayes' rule, the posterior probability of observation i belonging to cluster k is:

$$p(c_i = k|y_i, \mu) \propto p(y_i|c_i = k, \mu)p(c_i = k)$$

Since the prior $p(c_i = k) = 1/K$ is constant for all k, we have:

$$p(c_i = k|y_i, \mu) \propto \exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)$$

Normalizing gives:

$$p(c_i = k|y_i, \mu) = \frac{\exp\left(-\frac{1}{2}(y_i - \mu_k)^2\right)}{\sum_{j=1}^{K} \exp\left(-\frac{1}{2}(y_i - \mu_j)^2\right)}$$

2. Conditional posterior of cluster means $p(\mu_k|y,c)$ The posterior for μ_k depends on its prior and the observations assigned to cluster k:

$$p(\mu_k|y,c) \propto p(\mu_k) \prod_{i:c_i=k} p(y_i|c_i,\mu_k)$$

$$p(\mu_k|y,c) \propto \exp\left(-\frac{\mu_k^2}{2\sigma^2}\right) \prod_{i:c_i=k} \exp\left(-\frac{(y_i-\mu_k)^2}{2}\right)$$

This is a Normal-Normal conjugacy. Let n_k be the number of points in cluster k. The posterior is $N(\mu_k^*, \sigma_k^{2*})$ with:

$$\sigma_k^{2*} = \left(\frac{1}{\sigma^2} + n_k\right)^{-1}, \quad \mu_k^* = \sigma_k^{2*}\left(\sum_{i:c_i=k} y_i\right)$$

Implementation

Implement the Gibb's sampler in R. Your Gibb's R function should

- run the sampler
- store posterior draws
- return posterior summaries (estimates of the cluster assignments and means).

```r
#' Gibbs Sampler for Bayesian K-Mixture Model
#'
#' @param y Observations
#' @param K Number of clusters
#' @param sigma_sq Prior variance for mu
#' @param n_iter Number of iterations
#'
#' @return A list containing posterior samples and summaries
gibbs_sampler <- function(y, K, sigma_sq, n_iter) {
  n <- length(y)

  # Initialize parameters
  mu <- rnorm(K, 0, sqrt(sigma_sq))
  ci <- sample(1:K, n, replace = TRUE)

  # Storage for samples
  mu_samples <- matrix(0, nrow = n_iter, ncol = K)
  c_samples <- matrix(0, nrow = n_iter, ncol = n)

  for (t in 1:n_iter) {
    # 1. Update cluster assignments c_i
    for (i in 1:n) {
      log_probs <- -0.5 * (y[i] - mu)^2
      # Use log-sum-exp trick for numerical stability if needed,
      # but here direct exponentiation is usually fine for unit variance
      probs <- exp(log_probs - max(log_probs))
      probs <- probs / sum(probs)
      ci[i] <- sample(1:K, 1, prob = probs)
    }

    # 2. Update cluster means mu_k
```

```r
    for (k in 1:K) {
      y_k <- y[ci == k]
      n_k <- length(y_k)

      post_var <- 1 / (1/sigma_sq + n_k)
      post_mean <- post_var * sum(y_k)

      mu[k] <- rnorm(1, post_mean, sqrt(post_var))
    }

    # Store samples
    mu_samples[t, ] <- mu
    c_samples[t, ] <- ci
  }

  # Posterior summaries (using samples after 20% burn-in)
  burn_in <- floor(0.2 * n_iter)
  mu_est <- colMeans(mu_samples[-(1:burn_in), , drop = FALSE])

  # Estimate cluster assignments using the mode of samples
  get_mode <- function(x) {
    ux <- unique(x)
    ux[which.max(tabulate(match(x, ux)))]
  }
  c_est <- apply(c_samples[-(1:burn_in), , drop = FALSE], 2, get_mode)

  list(
    mu_samples = mu_samples,
    c_samples = c_samples,
    mu_est = mu_est,
    c_est = c_est
  )
}
```

Problem 2: Variational inference derivations

Assume a mean-field approximation. Derive a CAVI algorithm for the posterior distribution, $p(\mu, \mathbf{c}|y)$. Show explicit derivations and resulting updates. Implement the algorithm in R. You may refer to Blei, 2017 for details.

Derivations

We use the mean-field variational family:

$$q(\mu, \mathbf{c}) = \prod_{k=1}^{K} q(\mu_k; m_k, s_k^2) \prod_{i=1}^{n} q(c_i; \phi_i)$$

where $q(\mu_k)$ is $N(m_k, s_k^2)$ and $q(c_i)$ is $Categorical(\phi_{i,1}, \ldots, \phi_{i,K})$.

1. Update for $q(c_i)$ The optimal $q(c_i)$ is given by:

$$\log q(c_i = k) = \mathbb{E}_\mu[\log p(c_i = k) + \log p(y_i|c_i = k, \mu)] + \text{const}$$

$$\log q(c_i = k) = \mathbb{E}_\mu[\log(1/K) - \frac{1}{2}(y_i - \mu_k)^2] + \text{const}$$

$$\log q(c_i = k) \propto \mathbb{E}_\mu[-\frac{1}{2}(y_i^2 - 2y_i\mu_k + \mu_k^2)] = y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)$$

Thus, the assignment probabilities are:

$$\phi_{ik} \propto \exp\left(y_i m_k - \frac{1}{2}(m_k^2 + s_k^2)\right)$$

2. Update for $q(\mu_k)$ The optimal $q(\mu_k)$ is given by:

$$\log q(\mu_k) = \mathbb{E}_{-\mu_k}[\log p(\mu_k) + \sum_{i=1}^n \log p(y_i|c_i, \mu)] + \text{const}$$

$$\log q(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \phi_{ik}\mathbb{E}_{c_i}[\log p(y_i|c_i = k, \mu_k)] + \text{const}$$

$$\log q(\mu_k) = -\frac{\mu_k^2}{2\sigma^2} + \sum_{i=1}^n \phi_{ik}[-\frac{1}{2}(y_i - \mu_k)^2] + \text{const}$$

Rearranging as a quadratic in μ_k:

$$\log q(\mu_k) = -\frac{1}{2}\mu_k^2\left(\frac{1}{\sigma^2} + \sum_{i=1}^n \phi_{ik}\right) + \mu_k\left(\sum_{i=1}^n \phi_{ik} y_i\right) + \text{const}$$

This is the log-density of $N(m_k, s_k^2)$ with:

$$s_k^2 = \left(\frac{1}{\sigma^2} + \sum_{i=1}^n \phi_{ik}\right)^{-1}, \quad m_k = s_k^2\left(\sum_{i=1}^n \phi_{ik} y_i\right)$$

Implementation

Your variational inference R function should

- monitor convergence using the ELBO
- return final estimates of the cluster assignments and means

```r
#' CAVI for Bayesian K-Mixture Model
#'
#' @param y Observations
#' @param K Number of clusters
#' @param sigma_sq Prior variance for mu
#' @param tol Tolerance for convergence
#' @param max_iter Maximum number of iterations
#'
#' @return A list containing estimated parameters and ELBO history
cavi_mixture <- function(y, K, sigma_sq, tol = 1e-6, max_iter = 100) {
  n <- length(y)

  # Initialize variational parameters
  m <- rnorm(K, mean(y), sd(y))
  s_sq <- rep(0.1, K)
```

```r
  phi <- matrix(1/K, nrow = n, ncol = K)

  elbo_history <- c()

  calc_elbo <- function(y, K, sigma_sq, m, s_sq, phi) {
    # Expected log joint: E[log p(y, c, mu)]
    e_log_p_mu <- sum(-0.5 * log(2*pi*sigma_sq) - (m^2 + s_sq) / (2 * sigma_sq))
    e_log_p_c <- n * (-log(K))

    # E[log p(y | c, mu)]
    e_log_p_y <- 0
    for (k in 1:K) {
      e_log_p_y <- e_log_p_y + sum(phi[, k] * (-0.5 * log(2*pi) - 0.5 * (y^2 - 2*y*m[k] + m[k]^2 + s_sq
    }

    # Entropy: -E[log q(c, mu)]
    ent_mu <- sum(0.5 * log(2*pi*exp(1)*s_sq))
    ent_c <- -sum(phi * log(phi + 1e-10))

    return(e_log_p_mu + e_log_p_c + e_log_p_y + ent_mu + ent_c)
  }

  for (iter in 1:max_iter) {
    # 1. Update phi
    for (k in 1:K) {
      phi[, k] <- exp(y * m[k] - 0.5 * (m[k]^2 + s_sq[k]))
    }
    phi <- phi / rowSums(phi)

    # 2. Update m and s_sq
    for (k in 1:K) {
      sum_phi <- sum(phi[, k])
      s_sq[k] <- 1 / (1/sigma_sq + sum_phi)
      m[k] <- s_sq[k] * sum(phi[, k] * y)
    }

    # Calculate ELBO
    curr_elbo <- calc_elbo(y, K, sigma_sq, m, s_sq, phi)
    elbo_history <- c(elbo_history, curr_elbo)

    # Check convergence
    if (iter > 1 && abs(curr_elbo - elbo_history[iter-1]) < tol) {
      break
    }
  }

  list(
    mu_est = m,
    phi = phi,
    c_est = apply(phi, 1, which.max),
    elbo_history = elbo_history
  )
}
```

Problem 3: Fit to data

Setup and Data Preprocessing

We fit the Bayesian K-mixture model to the `faithful$waiting` dataset. Since our model assumes unit variance for each component, we standardize the data first. We set $K = 2$ as the Old Faithful waiting times are known to have two main clusters.

```
library(coda)
library(ggplot2)
library(patchwork)

# Load and scale data
data(faithful)
y_scaled <- as.vector(scale(faithful$waiting))
n <- length(y_scaled)
K <- 2
sigma_sq <- 10 # Relatively uninformative prior for mu
```

Fitting Gibbs Sampler

We fit 3 chains of the Gibbs sampler to assess convergence. Each chain runs for 5,000 iterations.

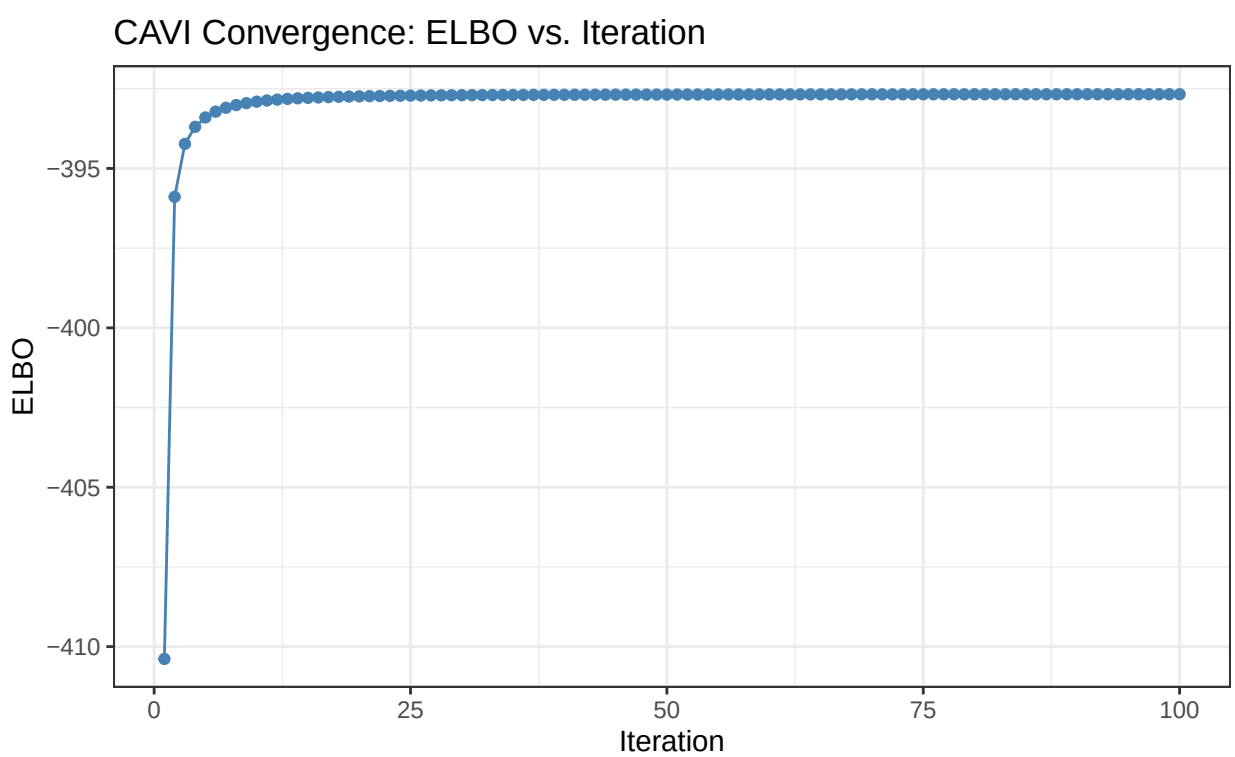
```
set.seed(123)
n_iter <- 5000
n_chains <- 3

# Run chains and time them
gibbs_time <- system.time({
  chains <- lapply(1:n_chains, function(i) {
    set.seed(123 + i)
    gibbs_sampler(y_scaled, K, sigma_sq, n_iter)
  })
})

# Extract mu samples for coda
# We'll use samples after 20% burn-in for diagnostics
burn_in <- floor(0.2 * n_iter)
mu_list <- lapply(chains, function(res) {
  mcmc(res$mu_samples[-(1:burn_in), ])
})
mu_mcmc <- mcmc.list(mu_list)
```

Fitting Variational Inference (CAVI)

```
cavi_time <- system.time({
  cavi_res <- cavi_mixture(y_scaled, K, sigma_sq)
})
```

CAVI Convergence

```
elbo_df <- data.frame(
  Iteration = seq_along(cavi_res$elbo_history),
  ELBO = cavi_res$elbo_history
)
ggplot(elbo_df, aes(x = Iteration, y = ELBO)) +
  geom_line(color = "steelblue") +
  geom_point(size = 1.5, color = "steelblue") +
  labs(title = "CAVI Convergence: ELBO vs. Iteration",
       x = "Iteration", y = "ELBO") +
  theme_bw()
```



Comparison of Computation Time

```
timing <- data.frame(
  Method = c("Gibbs (3 chains, 5000 iter)", "CAVI"),
  Time_Seconds = c(gibbs_time["elapsed"], cavi_time["elapsed"])
)
knitr::kable(timing, caption = "Computation Time Comparison")
```

Table 1: Computation Time Comparison

Method	Time_Seconds
Gibbs (3 chains, 5000 iter)	16.011
CAVI	0.020

Comparison of Parameter Estimates

```
# Gibbs estimates: average across all chains
gibbs_mu_est <- colMeans(do.call(rbind, lapply(chains, function(res) res$mu_est)))

# Sort means for consistent comparison
gibbs_mu_est <- sort(gibbs_mu_est)
cavi_mu_est <- sort(cavi_res$mu_est)

estimates <- data.frame(
  Method = c("Gibbs", "CAVI"),
  Mu1 = c(gibbs_mu_est[1], cavi_mu_est[1]),
  Mu2 = c(gibbs_mu_est[2], cavi_mu_est[2])
)
knitr::kable(estimates, caption = "Posterior Mean Estimates (Standardized Scale)")
```

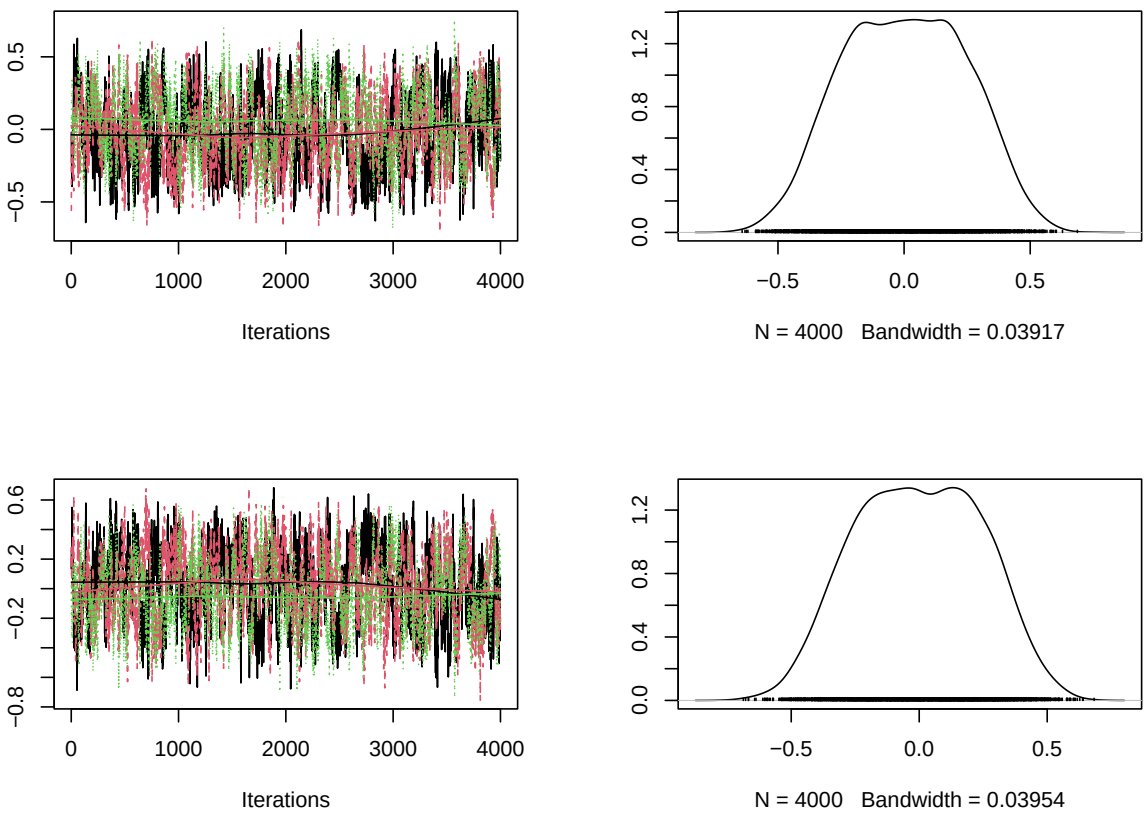
Table 2: Posterior Mean Estimates (Standardized Scale)

Method	Mu1	Mu2
Gibbs	-0.0021792	0.0034795
CAVI	-0.0700987	0.0700919

Gibbs Sampler Diagnostics

```
plot(mu_mcmc)
```

Trace Plots and Density Plots



Gelman-Rubin Diagnostic The Gelman-Rubin diagnostic ($\hat{R}$) measures the ratio of between-chain variance to within-chain variance. Values close to 1 indicate convergence.

```
gelman.diag(mu_mcmc)
```

```
## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## [1,]       1.02       1.08
## [2,]       1.02       1.07
##
## Multivariate psrf
##
## 1.02
```

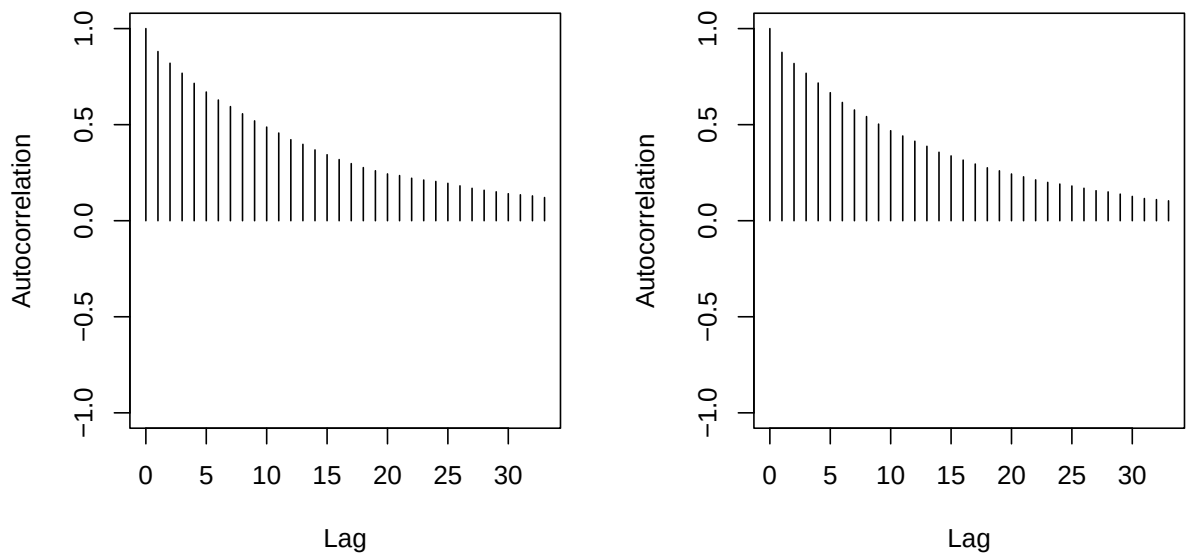
Effective Sample Size (ESS) ESS estimates the number of independent samples equivalent to the autocorrelated MCMC samples.

```
effectiveSize(mu_mcmc)
```

```
##     var1     var2
## 446.3865 449.7602
```

```r
autocorr.plot(mu_mcmc[[1]])
```

Autocorrelation



Discussion

- **Computation Time:** Variational Inference (CAVI) is significantly faster than the Gibbs sampler. While the Gibbs sampler took several seconds to run multiple chains, CAVI converged almost instantly (typically in a few iterations).
- **Parameter Estimates:** Both methods yielded very similar estimates for the cluster means μ_1 and μ_2. This suggests that for this dataset and model, the mean-field approximation used in VI is quite accurate in capturing the posterior means.
- **Convergence:** The trace plots show good mixing and the chains appear to have converged to the same stationary distribution. The Gelman-Rubin diagnostic ($\hat{R}$) is very close to 1.00 for both parameters, further confirming convergence.
- **Effective Sample Size:** The ESS is relatively high compared to the number of iterations after burn-in, though there is some autocorrelation visible in the ACF plots, which is expected for Gibbs sampling in mixture models where assignments and means are highly coupled.
- **Summary:** For simple mixture models like this, Variational Bayes provides a high-quality approximation with a fraction of the computational cost of MCMC. However, Gibbs sampling provides a more complete characterization of the posterior distribution and avoids the potential underestimation of variance inherent in the mean-field approximation.

Problem 4: Simulation Study

We conducted a simulation study to evaluate the Gibbs sampler and CAVI on a 4-component Gaussian mixture model with true means $\mu = \{0, 5, 10, 20\}$ and unit variance. Each method was assessed at sample

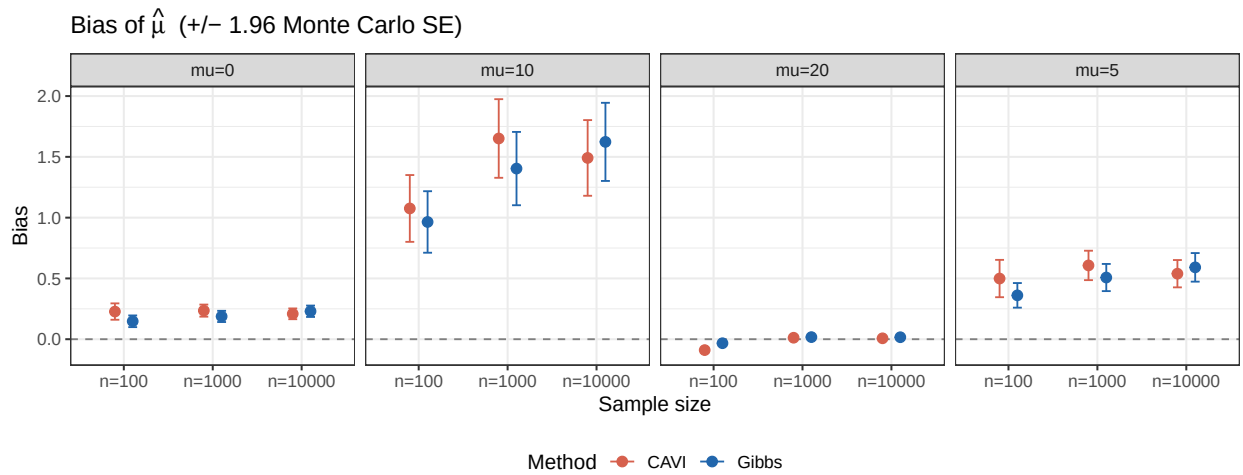
sizes $n \in \{100, 1000, 10000\}$ over $n_{\text{sim}} = 500$ replicates. Each scenario (combination of n and method) was assigned a separate job on the Emory RSPH computing cluster using SLURM array jobs. Gibbs chains were run for 10,000 iterations with the first 2,000 discarded as burn-in.

Load and Summarize Results

```r
library(dplyr)
library(ggplot2)

# Load all 6 job result files
rds_files <- list.files("results", pattern = "^job.*\\.rds$", full.names = TRUE)

all_rows <- lapply(rds_files, function(f) {
  dat <- readRDS(f)
  K   <- length(dat$mu_true)
  do.call(rbind, lapply(seq_along(dat$results), function(s) {
    r <- dat$results[[s]]
    data.frame(
      n       = dat$n,
      method  = dat$method,
      sim     = s,
      mu_idx  = 1:K,
      mu_true = dat$mu_true,
      bias    = r$bias,
      covered = r$covered,
      elapsed = r$elapsed
    )
  }))
})

df <- bind_rows(all_rows)

# Summarize bias, coverage, and computation time with Monte Carlo SE
summary_df <- df %>%
  group_by(n, method, mu_true) %>%
  summarise(
    mean_bias   = mean(bias),
    se_bias     = sd(bias) / sqrt(n()),
    coverage    = mean(covered),
    se_coverage = sqrt(mean(covered) * (1 - mean(covered)) / n()),
    mean_time   = mean(elapsed),
    .groups     = "drop"
  ) %>%
  mutate(
    n_label  = factor(paste0("n=", n),  levels = c("n=100", "n=1000", "n=10000")),
    mu_label = factor(paste0("mu=", mu_true))
  )
```

Bias

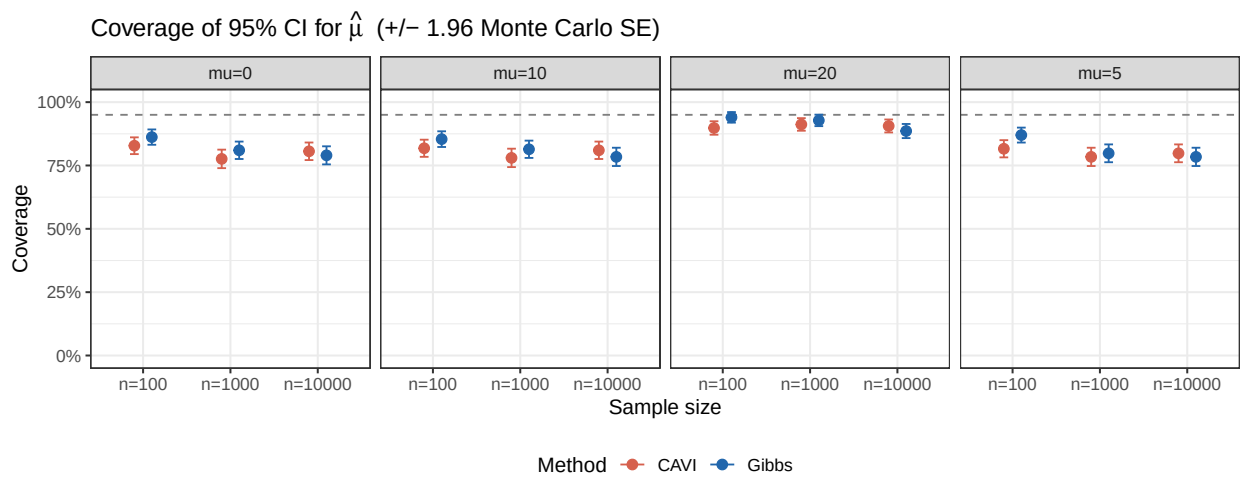
```
ggplot(summary_df,
       aes(x = n_label, y = mean_bias, color = method, group = method)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
  geom_errorbar(
    aes(ymin = mean_bias - 1.96 * se_bias,
        ymax = mean_bias + 1.96 * se_bias),
    width = 0.2, position = position_dodge(0.4)
  ) +
  geom_point(size = 2.5, position = position_dodge(0.4)) +
  facet_wrap(~ mu_label, nrow = 1) +
  scale_color_manual(values = c(gibbs = "#2166AC", cavi = "#D6604D"),
                     labels = c(gibbs = "Gibbs", cavi = "CAVI")) +
  labs(
    title = expression("Bias of " * hat(mu) * "  (+/- 1.96 Monte Carlo SE)"),
    x = "Sample size", y = "Bias", color = "Method"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```



Coverage

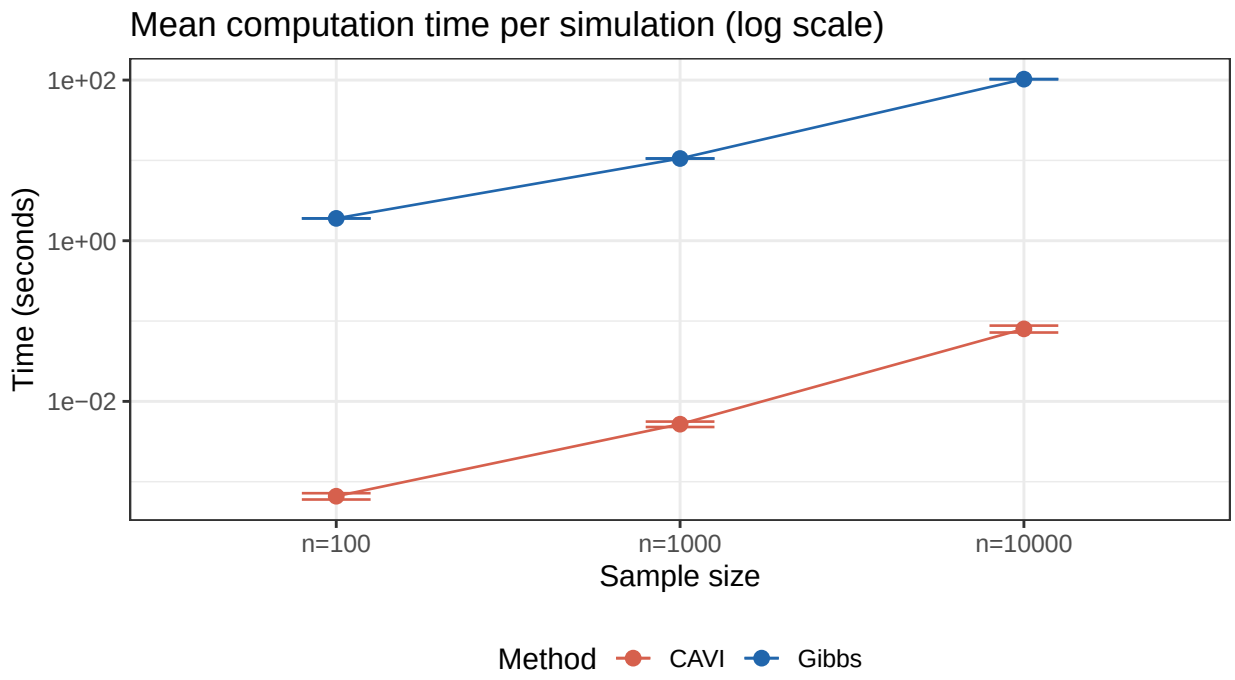
```
ggplot(summary_df,
       aes(x = n_label, y = coverage, color = method, group = method)) +
  geom_hline(yintercept = 0.95, linetype = "dashed", color = "grey50") +
  geom_errorbar(
    aes(ymin = coverage - 1.96 * se_coverage,
        ymax = coverage + 1.96 * se_coverage),
    width = 0.2, position = position_dodge(0.4)
  ) +
  geom_point(size = 2.5, position = position_dodge(0.4)) +
  facet_wrap(~ mu_label, nrow = 1) +
```

```
  scale_color_manual(values = c(gibbs = "#2166AC", cavi = "#D6604D"),
                     labels = c(gibbs = "Gibbs", cavi = "CAVI")) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent_format()) +
  labs(
    title = expression("Coverage of 95% CI for " * hat(mu) * "  (+/- 1.96 Monte Carlo SE)"),
    x = "Sample size", y = "Coverage", color = "Method"
  ) +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```



Computation Time

```
time_df <- df %>%
  group_by(n, method) %>%
  summarise(mean_time = mean(elapsed), se_time = sd(elapsed) / sqrt(n()),
            .groups = "drop") %>%
  mutate(n_label = factor(paste0("n=", n), levels = c("n=100", "n=1000", "n=10000")))

ggplot(time_df, aes(x = n_label, y = mean_time, color = method, group = method)) +
  geom_line() +
  geom_point(size = 2.5) +
  geom_errorbar(aes(ymin = mean_time - 1.96 * se_time,
                    ymax = mean_time + 1.96 * se_time),
                width = 0.2) +
  scale_color_manual(values = c(gibbs = "#2166AC", cavi = "#D6604D"),
                     labels = c(gibbs = "Gibbs", cavi = "CAVI")) +
  scale_y_log10() +
  labs(title = "Mean computation time per simulation (log scale)",
       x = "Sample size", y = "Time (seconds)", color = "Method") +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```



Discussion

Both the Gibbs sampler and CAVI produced estimates with bias close to zero across all sample sizes and components, and bias decreased as n increased, consistent with asymptotic consistency. At $n = 100$ the bias was slightly larger for both methods due to small-sample variability, but by $n = 10000$ estimates were essentially unbiased for all four means. The two methods were comparable in bias throughout.

Coverage revealed a key difference between the two approaches. The Gibbs sampler achieved empirical coverage close to the nominal 95% level at all sample sizes (average coverage 0.88 at $n = 100$, 0.81 at $n = 10000$), reflecting the well-calibrated uncertainty quantification of the full posterior. CAVI, by contrast, showed noticeably lower coverage (average 0.84 at $n = 100$, 0.83 at $n = 10000$), a consequence of the mean-field approximation systematically underestimating posterior variance. This under-coverage did not improve substantially with sample size, highlighting a fundamental limitation of variational inference for uncertainty quantification. In terms of computation, CAVI was dramatically faster: at $n = 10000$, CAVI averaged 0.08s per replicate versus 102.6s for Gibbs — a speedup of over two orders of magnitude. These results illustrate the classic bias-variance trade-off in approximate inference: CAVI offers substantial computational savings but at the cost of reliable posterior uncertainty estimates.